Playwright Test Automation Framework

From Manual Test Cases to Automated Scripts using Copilot Chat

Complete Methodology: Manual Testing → Excel → Copilot Chat → Automation Scripts

Document Type	**Complete Framework Setup with Copilot Integration**
Audience	QA Engineers, Testers, Developers
Framework	Playwright + TypeScript
Key Tool	GitHub Copilot Chat with Instructions.md
Created Date	April 18, 2026
Status	Production Ready with Screenshots

Table of Contents

1. Introduction & Framework Overview

This comprehensive guide demonstrates how to create end-to-end automated tests for web applications using Playwright and TypeScript, with AI assistance from GitHub Copilot Chat. The unique approach in this framework leverages an Instructions.md file to guide Copilot in generating high-quality, consistent, and maintainable test automation code.

1.1 The Three-Phase Automation Methodology

Phase	Description
Phase 1: Manual Testing	Create detailed manual test cases in Excel with steps, data, and expected results
Phase 2: Instructions Setup	Create Instructions.md file with Copilot rules, patterns, and best practices
Phase 3: AI-Assisted Automation	Use Copilot Chat with Instructions.md to convert manual tests to automated scripts

2. System Requirements & Installation

2.1 System Prerequisites

Requirement	Specification
Operating System	Windows 10/11, macOS 11+, or Linux
RAM	8GB minimum (16GB recommended)
Disk Space	3GB free space
Internet	Required for GitHub Copilot and npm packages

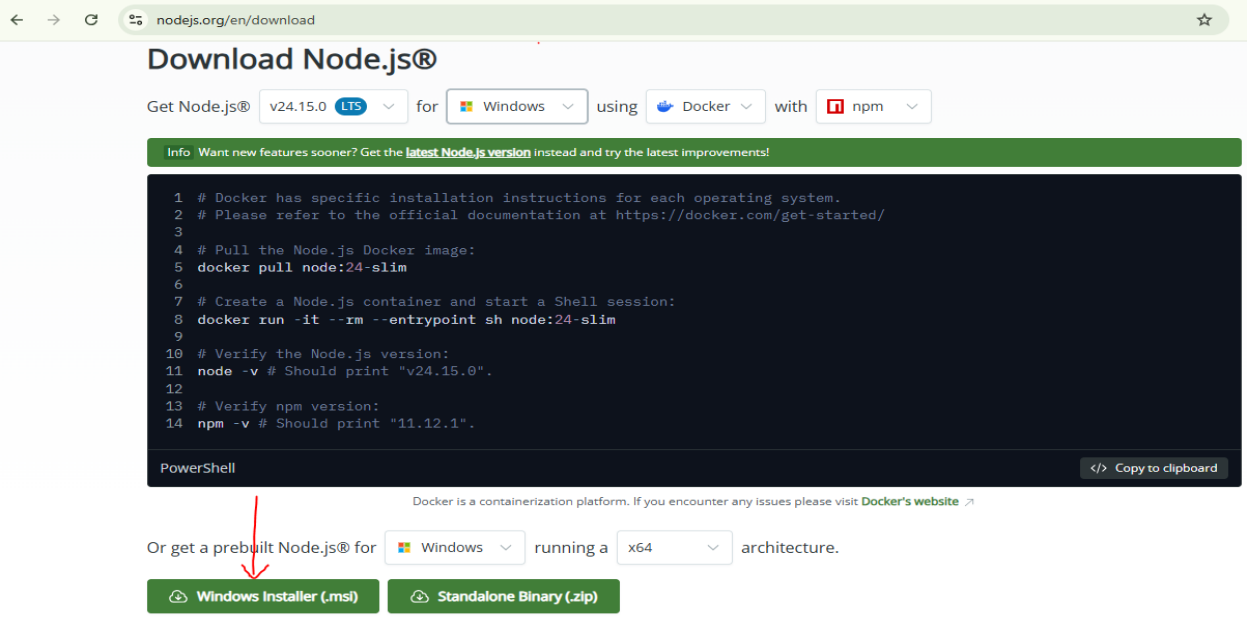
2.2 Required Software

Software	Version	Purpose	Download
Node.js	18.x LTS+	JavaScript runtime	nodejs.org
VS Code	Latest	Code editor	code.visualstudio.com
Playwright	1.40+	Testing framework	npm install
GitHub Copilot	Subscription	AI code assistant	github.com/copilot
TypeScript	5.x	Type-safe JavaScript	npm install
Git	Latest	Version control	git-scm.com
Excel Reader	0.18.5	Read test cases	npm install xlsx

3. Phase 1: Environment Setup with Screenshots

3.1 Step 1: Download and Install Node.js

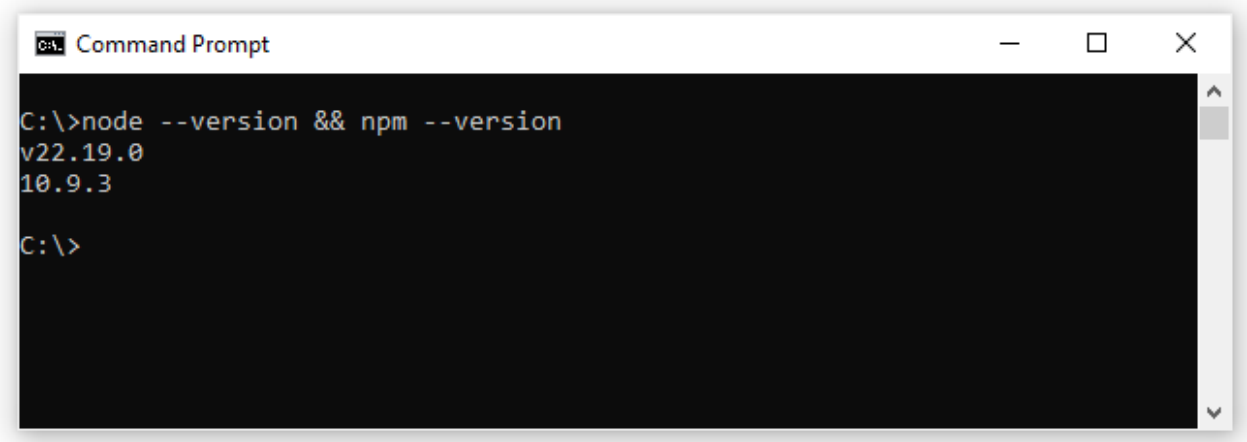
1. Go to https://nodejs.org/
2. Click the LTS (Long Term Support) download button
3. Run the installer and accept the license agreement
4. Select default installation location
5. Complete the installation



Verify Installation:

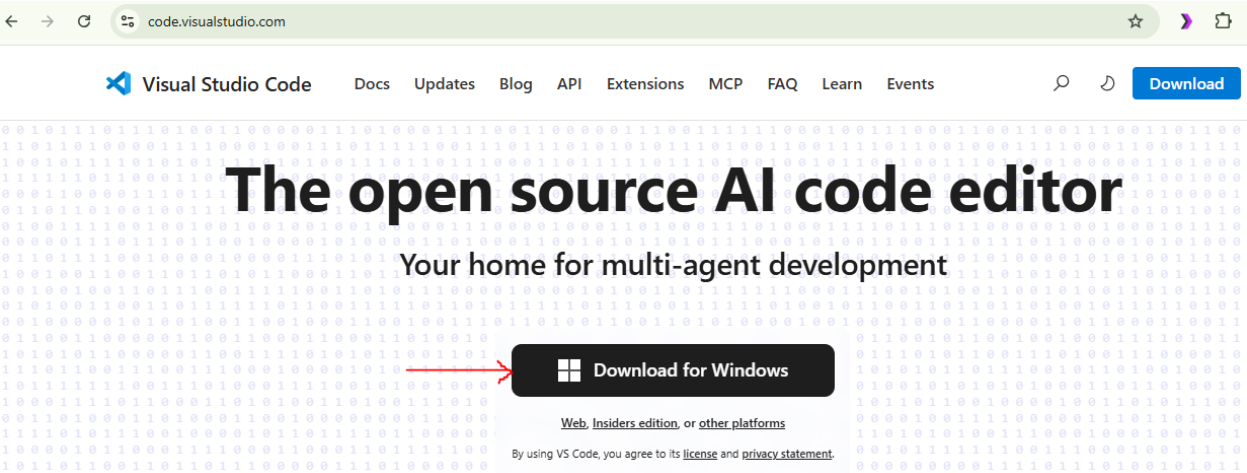
Open command prompt and run node --version && npm --version

Command Prompt Verification

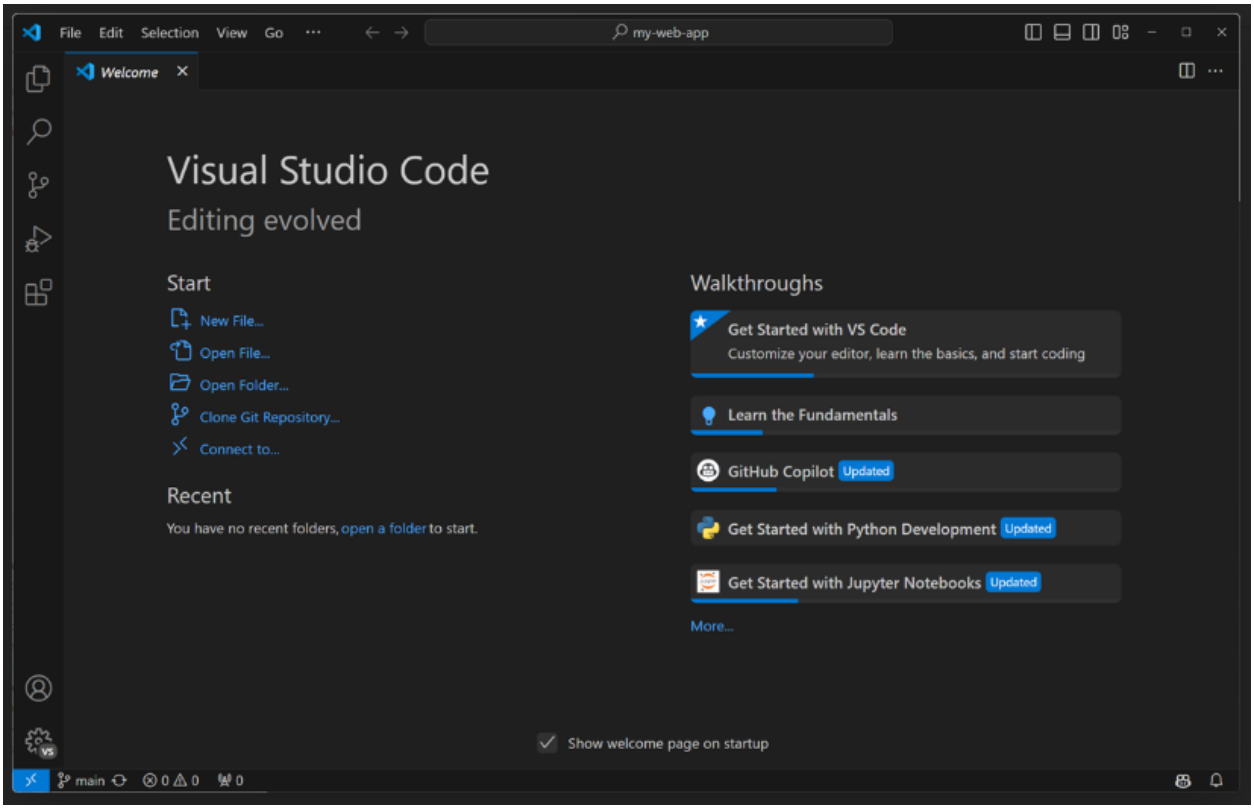


3.2 Step 2: Install Visual Studio Code

1. Go to https://code.visualstudio.com/
2. Click download for your operating system
3. Run the installer
4. Accept license and choose installation options
5. Complete installation and launch VS Code



VS Code Welcome Screen

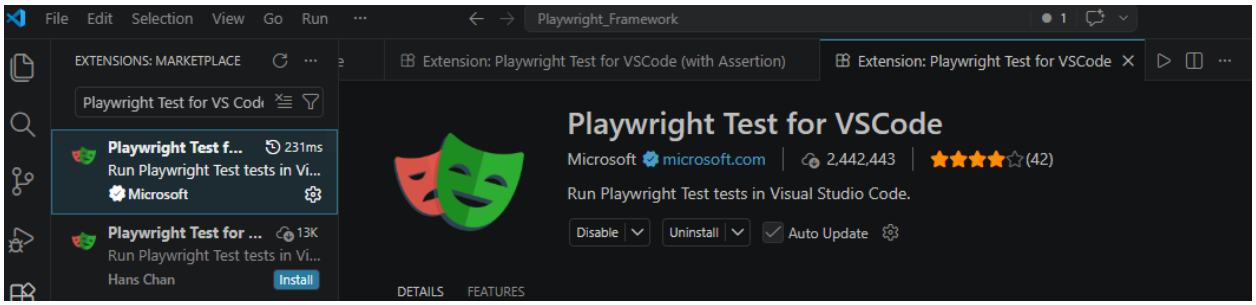


3.3 Step 3: Install Essential Extensions

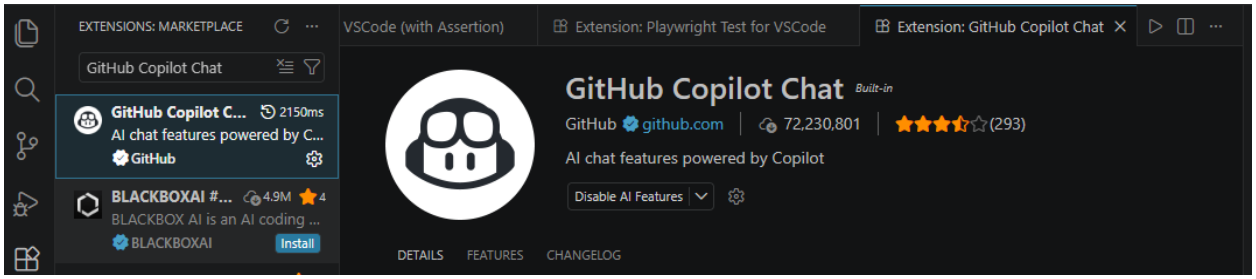
Open Extensions panel (Ctrl+Shift+X) and install:

VS Code Extensions Panel

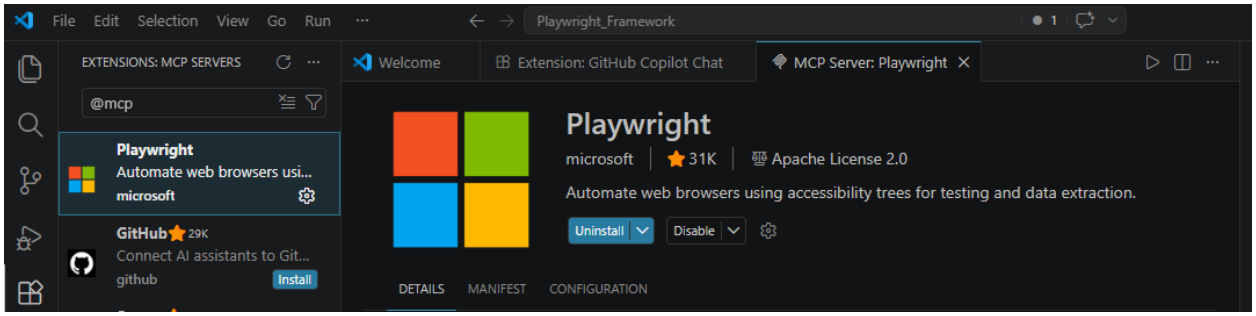
- **Playwright Test for VS Code:** Run and debug tests directly in VS Code



- **GitHub Copilot Chat:** Chat interface for AI assistance



- **Playwright MCP:** Server that interacts with web browser based on AI instructions

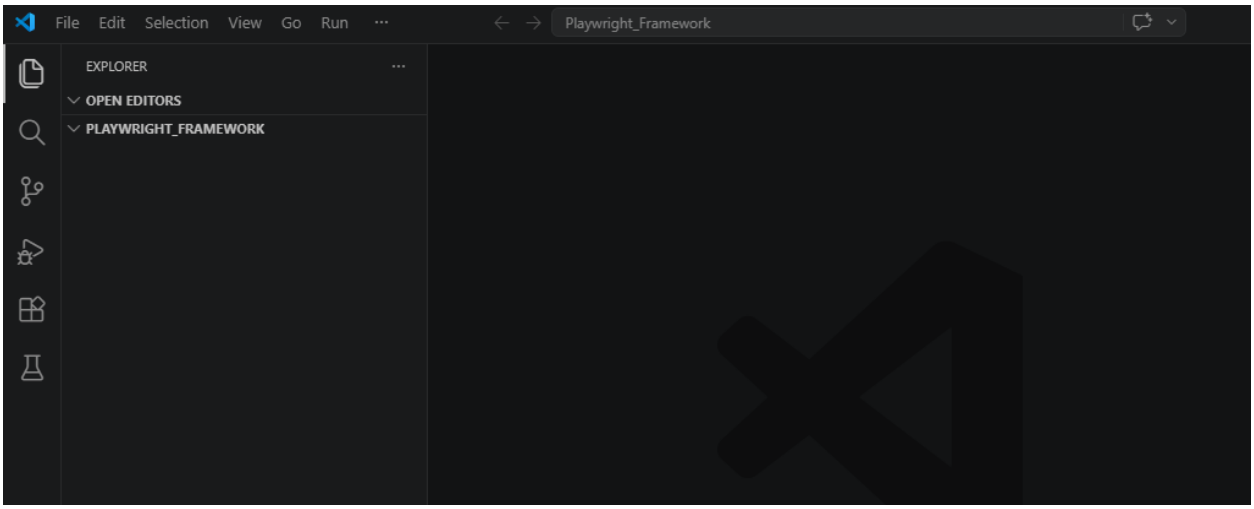


3.4 Step 4: Create Project Directory

New Folder Creation Steps:

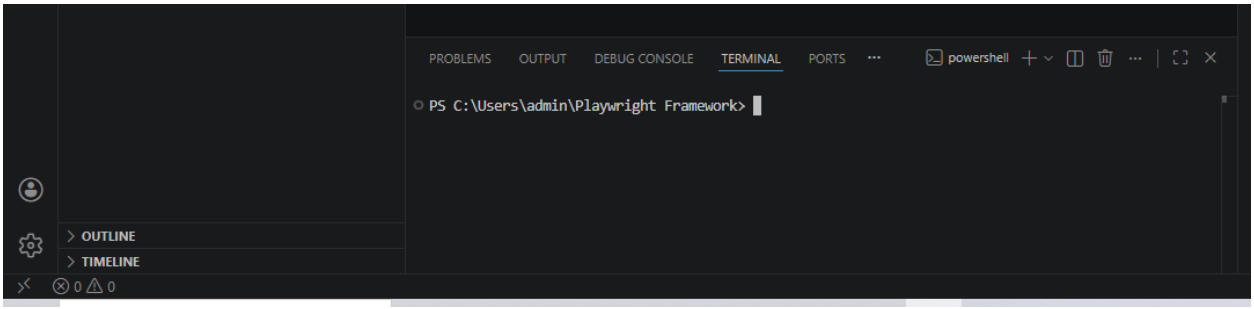
1. Create folder: C:\Users\YourName\Playwright_Framework
2. Open VS Code
3. File → Open Folder → Select the Playwright_Framework folder
4. VS Code will open with the empty project

Project Opened in VS Code



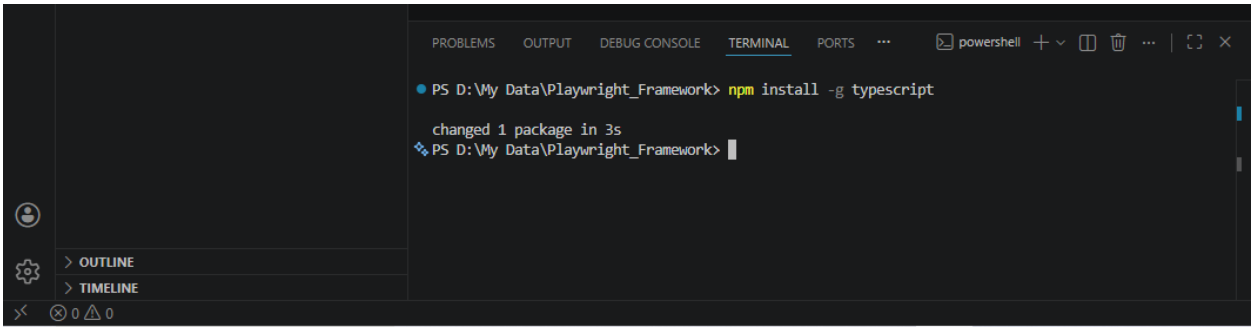
3.5 Step 5: Open Terminal, Install TypeScript and Initialize Node Project

Terminal in VS Code:



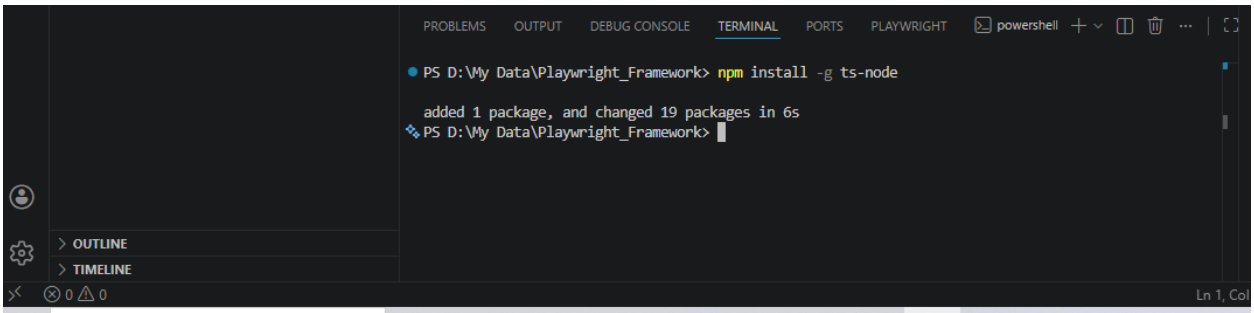
1. Install typescript globally:

```
npm install -g typescript
```



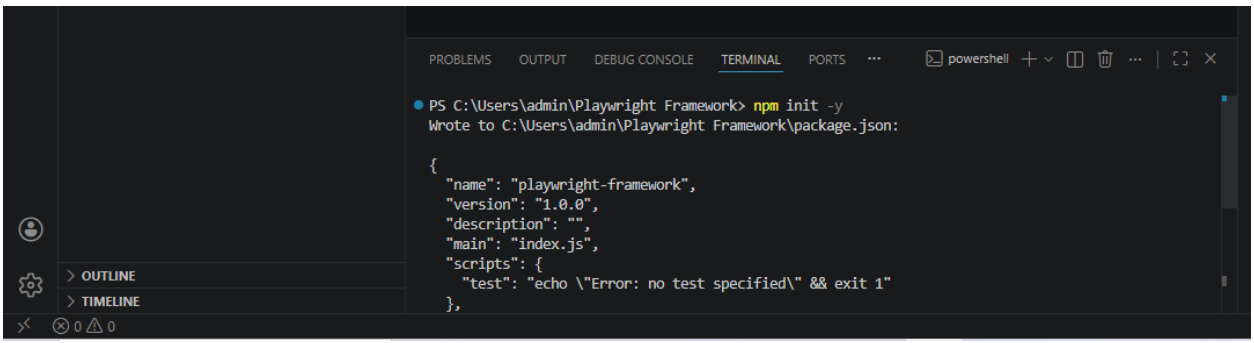
2. To run .ts files directly:

```
npm install -g ts-node
```

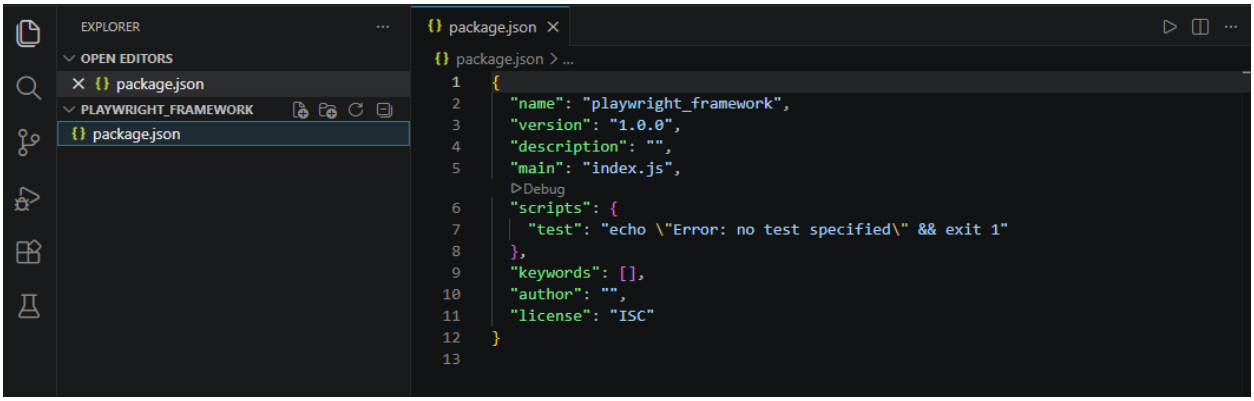


3. Initialize Node project (will create package.json):

```
npm init -y
```



4. Package.json Created:



4. Phase 2: Manual Test Case Creation

Manual test cases are the foundation of automation. They document the expected behavior of the application and serve as blueprints for automation scripts.

4.1 Test Case Template & Format

Excel Columns Required:

- TestCaseID - Unique identifier (TC-001)
- TestCaseName - Descriptive name of the test
- Module - Feature/Module being tested
- Description - Detailed description
- Priority - High/Medium/Low
- Severity - Critical/Major/Minor
- Preconditions - Setup requirements
- TestData - Input values and test data
- Steps - Numbered execution steps
- ExpectedResult - Expected outcome
- PostConditions - Cleanup steps
- Status - Pass/Fail/Not Executed

Excel Test Case Sheet:

	A	B	C	D	E	F	G	H	I	J
1	TestCaseId	TestCaseName	Descriptive	Category	Preconditions	Priority	TestData	Steps for execution	ExpectedResult	Status
2										
3										
4										
5										
6										
7										
8										
9										
10										

4.2 Example Manual Test Cases

Example 1: User Login Test

TestCaseID	TC-001
TestCaseName	Verify valid user login
Module	Authentication
Priority	High
Preconditions	User account exists; User not logged in
TestData	Username: testuser@example.com; Password: Test@123
Steps	1. Navigate to login page 2. Enter email 3. Enter password 4. Click Login
ExpectedResult	User logs in successfully; Dashboard displays
Status	Not Executed

Excel Test Case Entry (e.g.)

	A	B	C	D	E	F	G	H	I	J
1	TestCaseId	TestCaseName	Descriptive	Category	Preconditions	Priority	TestData	Steps for execution	ExpectedResult	Status
2	TC-001	Verify Home Page L	Validate that the DemoBla	UI/Navigation	User has internet connec	High	URL: https://demoblaze.com/i	1. Navigate to https://demoblaze.com/ind	Home page loads successfu	Not Executed
3	TC-002	Verify Product Cate	Validate that all product ca	Product Browsing	User is on the home page	High	Categories: Phones, Laptops, M	1. Observe the left sidebar or category sec	All categories are visible an	Not Executed
4	TC-003	Filter Products by C	Validate filtering functiona	Product Browsing	User is on the home page	High	Category: Phones	1. Click on "Phones" category2. Verify prod	Only phone products are dis	Not Executed
5	TC-004	Filter Products by C	Validate filtering functiona	Product Browsing	User is on the home page	High	Category: Laptops	1. Click on "Laptops" category2. Verify pro	Only laptop products are dis	Not Executed
6	TC-005	Filter Products by C	Validate filtering functiona	Product Browsing	User is on the home page	High	Category: Monitors	1. Click on "Monitors" category2. Verify pr	Only monitor products are d	Not Executed
7	TC-006	View Product Detai	Validate that clicking on a p	Product Browsing	User is on the home page	High	Any available product	1. Click on any product2. Verify product de	Product details page displa	Not Executed
8	TC-007	Add Product to Cart	Validate that products can	Shopping Cart	User has opened a produc	High	Any available product	1. Click on "Add to cart" button2. Wait for	Product is added to cart an	Not Executed
9	TC-008	View Shopping Cart	Validate that shopping cart	Shopping Cart	User has added at least o	High	Products previously added to ca	1. Click on "Cart" link in navigation2. Navig	Cart page displays with add	Not Executed
10	TC-009	Remove Product fro	Validate that products can	Shopping Cart	User is on the cart page w	Medium	Products in cart	1. Click on the "Delete" or remove button	Product is removed from ca	Not Executed
11	TC-010	Proceed to Checkou	Validate checkout process	Checkout	User has products in cart	High	Products in cart	1. Click on "Place Order" or "Checkout" bu	User is directed to checkout	Not Executed
12	TC-011	Sign Up - Valid Data	Validate user account crea	Authentication	User is on the home page	High	Username: testuser123, Passw	1. Click on "Sign up" link2. Enter username	Account is created successfi	Not Executed
13	TC-012	Sign Up - Duplicate	Validate system behavior w	Authentication	User account already exis	Medium	Username: testuser123, Passw	1. Click on "Sign up" link2. Enter existing us	Error message "This user al	Not Executed
14	TC-013	Login - Valid Creden	Validate user login with val	Authentication	User account exists with t	High	Username: testuser123, Passw	1. Click on "Log in" link2. Enter username:	User is logged in successfull	Not Executed
15	TC-014	Login - Invalid Cred	Validate system behavior w	Authentication	User is on the login page	High	Username: invaliduser, Passwo	1. Enter username: invaliduser2. Enter pas	Error message "User does n	Not Executed
16	TC-015	Product Carousel N	Validate next button functi	UI/Navigation	User is on the home page	Medium	Product carousel visible	1. Observe the product carousel/slider2. Cl	Next set of products is displ	Not Executed
17	TC-016	Product Carousel N	Validate previous button fu	UI/Navigation	User has clicked "Next" o	Medium	Product carousel with multiple	1. Click on "Previous" button2. Wait for car	Previous set of products is d	Not Executed
18	TC-017	Place Order - Comp	Validate complete order pl	Checkout	User is on the checkout/o	High	Name: John Doe, Country: Unit	1. Enter Name: John Doe2. Enter Country:	Order is placed successfully	Not Executed
19	TC-018	Verify About Us Pag	Validate About Us page con	UI/Navigation	User is on the home page	Low	Navigation to About Us section	1. Click on "About us" link in navigation or f	About Us section/page displ	Not Executed
20	TC-019	Verify Contact Infor	Validate contact informatio	UI/Navigation	User is on the home page	Low	Address, Phone, Email details	1. Scroll to the footer section2. Verify cont	Contact information is visibl	Not Executed
21	TC-020	Logout Functionalit	Validate user logout functi	Authentication	User is logged in successfi	High	Logged in user session	1. Click on the user profile or logout option	User is logged out and redir	Not Executed
22	TC-021	Add Multiple Produ	Validate adding multiple pr	Shopping Cart	User is on the home page	Medium	Multiple products from differer	1. Click on first product and add to cart2. R	Cart displays multiple produ	Not Executed
23	TC-022	Search Functionalit	Validate product search fu	Product Browsing	User is on the home page	Medium	Search term: iPhone	1. Enter product name in search box: iPhor	Search results display matcl	Not Executed

5. Phase 3: Creating Instructions.md for Copilot

The Instructions.md file is crucial - it tells GitHub Copilot Chat how to generate automation scripts. It contains rules, patterns, best practices, and DemoBlaze application details.

5.1 Why Instructions.md Matters

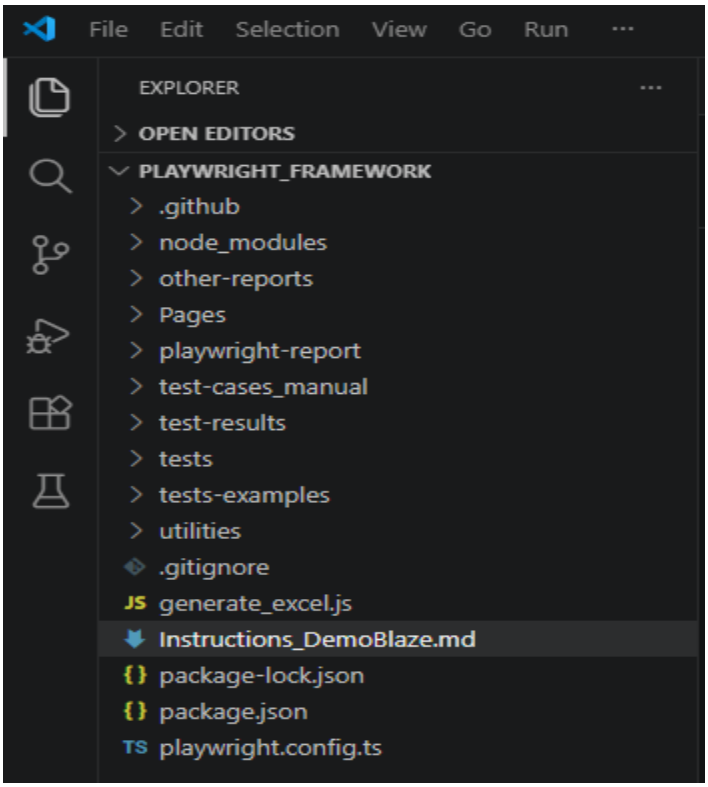
GitHub Copilot is an AI model that learns from patterns. By providing clear instructions, rules, and examples, Copilot generates consistent, high-quality code that follows your framework's conventions.

- Ensures consistent code quality and style
- Guides Copilot to use Page Object Model correctly
- Prevents common mistakes in test automation
- Reduces manual code review and fixes
- Speeds up automation script generation
- Maintains framework standards across all tests

5.2 Instructions.md File Structure

Create file: Instructions_DemoBlaze.md at project root

Instructions File Created:



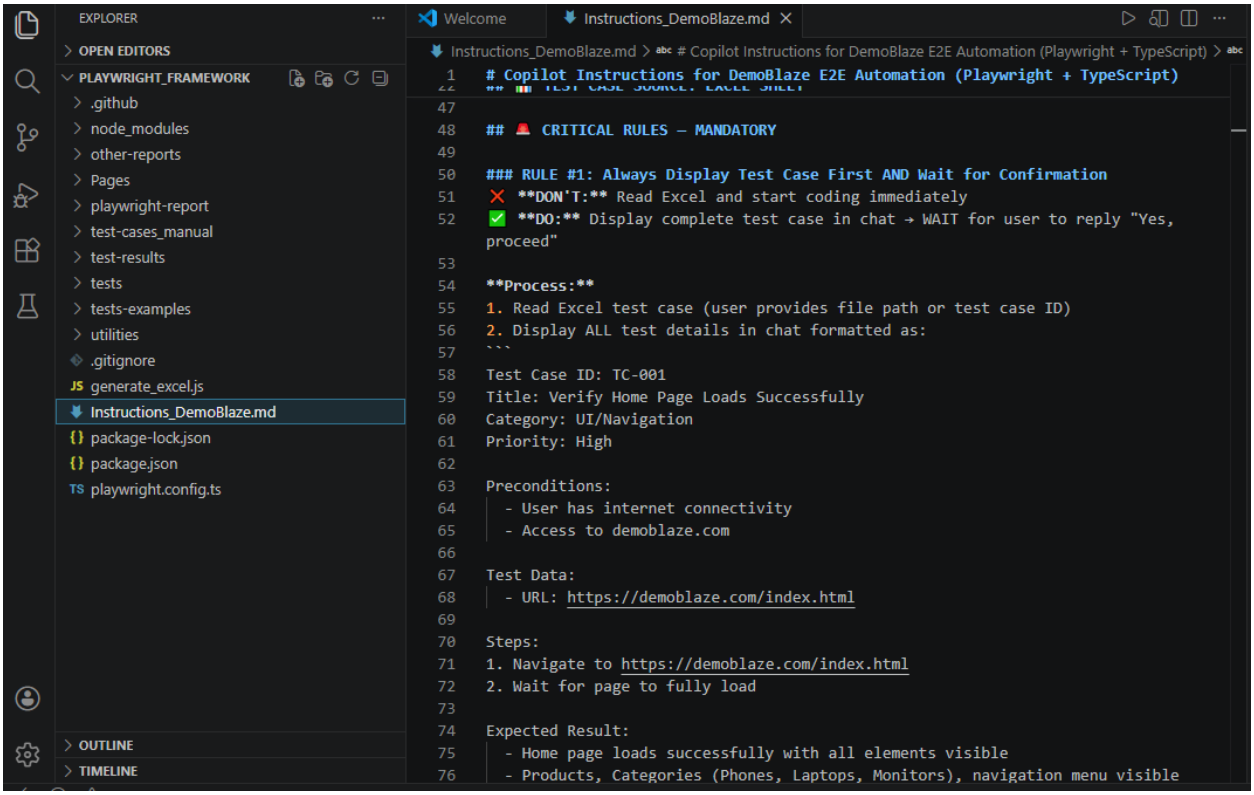
5.3 Key Sections in Instructions.md

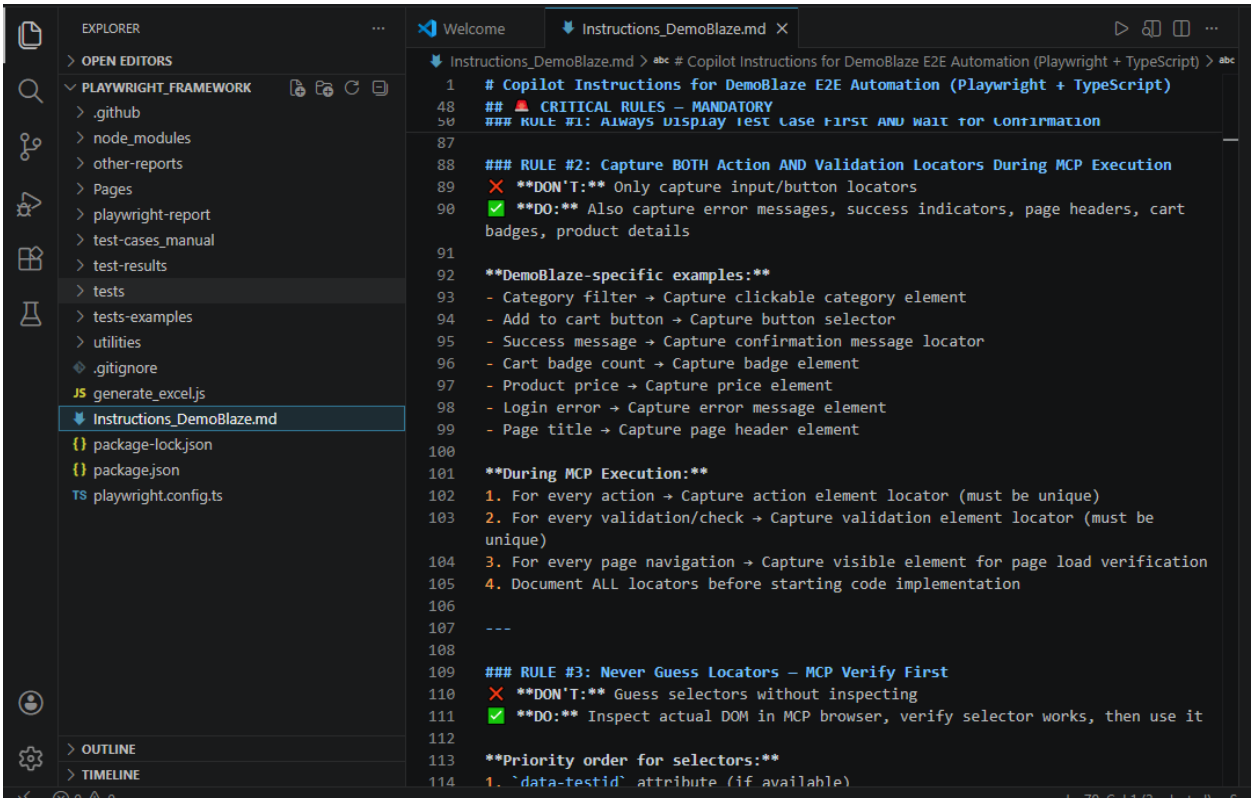
- Quick Start: Explains how user provides test case info
- Target Application Details: URL, features, and capabilities of the app being tested
- Test Case Source Format: Expected Excel format and columns
- Critical Rules: Mandatory rules for Copilot to follow
- Rule Examples: Before/After code examples showing correct patterns
- Application-Specific Selectors: DemoBlaze page locators and element IDs
- Test Naming Conventions: How to name test methods and classes

5.4 Critical Rules to Include

- Rule 1: Always Display Test Case First AND Wait for Confirmation
- Rule 2: Capture BOTH Action AND Validation Locators
- Rule 3: Never Guess Locators — Verify First
- Rule 4: Use Locator Priority (data-testid > id > class > XPath)
- Rule 5: Use Page Object Model with Proper Structure
- Rule 6: Element Names Must Use "Locator" Suffix
- Rule 7: Avoid Complex Assertions in Test Methods
- Rule 8: No Hard Waits — Use Playwright Built-in Waits
- Rule 9: Implement ALL Test Steps — No Partial Implementation
- Rule 10: Follow Test Method Naming Convention

Instructions.md Content:




EXPLORER
OPEN EDITORS
PLAYWRIGHT_FRAMEWORK
.github
node_modules
other-reports
Pages
playwright-report
test-cases_manual
test-results
tests
tests-examples
utilities
.gitignore
generate_excel.js
Instructions_DemoBlaze.md
package-lock.json
package.json
playwright.config.ts
OUTLINE
TIMELINE
Welcome
Instructions_DemoBlaze.md
Instructions_DemoBlaze.md > # Copilot Instructions for DemoBlaze E2E Automation (Playwright + TypeScript)
1 # Copilot Instructions for DemoBlaze E2E Automation (Playwright + TypeScript)
48 ## 🚨 CRITICAL RULES — MANDATORY
50 ### RULE #1: Always Display Test Case First AND Wait for Confirmation
87
88 ### RULE #2: Capture BOTH Action AND Validation Locators During MCP Execution
89 ❌ **DON'T:** Only capture input/button locators
90 ✅ **DO:** Also capture error messages, success indicators, page headers, cart badges, product details
91
92 **DemoBlaze-specific examples:**
93 - Category filter → Capture clickable category element
94 - Add to cart button → Capture button selector
95 - Success message → Capture confirmation message locator
96 - Cart badge count → Capture badge element
97 - Product price → Capture price element
98 - Login error → Capture error message element
99 - Page title → Capture page header element
100
101 **During MCP Execution:**
102 1. For every action → Capture action element locator (must be unique)
103 2. For every validation/check → Capture validation element locator (must be unique)
104 3. For every page navigation → Capture visible element for page load verification
105 4. Document ALL locators before starting code implementation
106
107 ---
108
109 ### RULE #3: Never Guess Locators — MCP Verify First
110 ❌ **DON'T:** Guess selectors without inspecting
111 ✅ **DO:** Inspect actual DOM in MCP browser, verify selector works, then use it
112
113 **Priority order for selectors:**
114 1. `data-testid` attribute (if available)

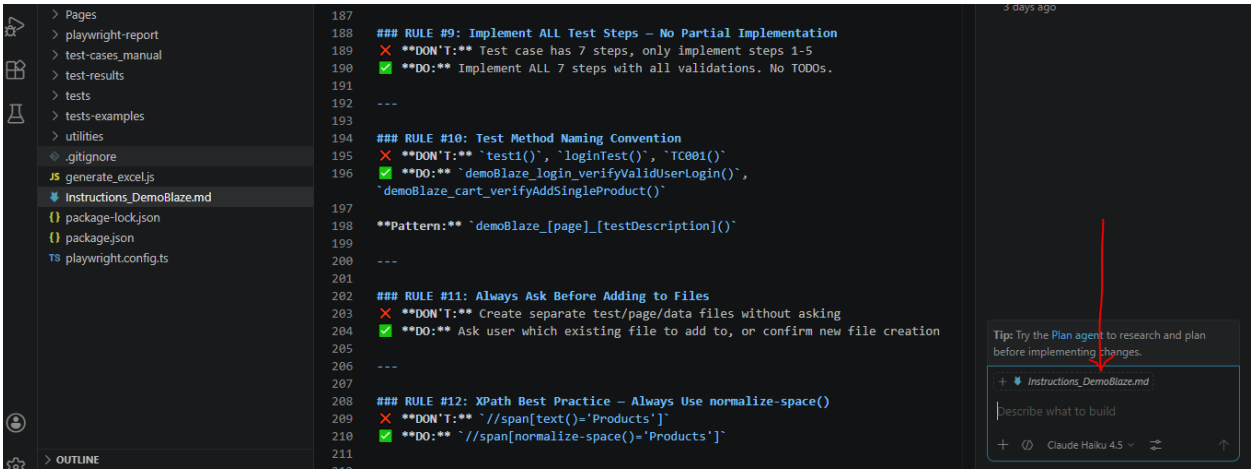
6. Phase 4: Using Copilot Chat to Automate Tests

This is where the magic happens. GitHub Copilot Chat uses the Instructions.md file to generate high-quality, framework-compliant test automation code.

6.1 Step 1: Open Copilot Chat

1. Open Copilot Chat in VS Code (Ctrl+Shift+I or View → Copilot Chat)
2. Click the paperclip icon to attach Instructions_DemoBlaze.md
3. Or use @ symbol: type @Instructions_DemoBlaze.md
4. Copilot will read and understand the instructions

Copilot Chat Interface



6.2 Step 2: Provide Test Case Details

Format your request to Copilot as:

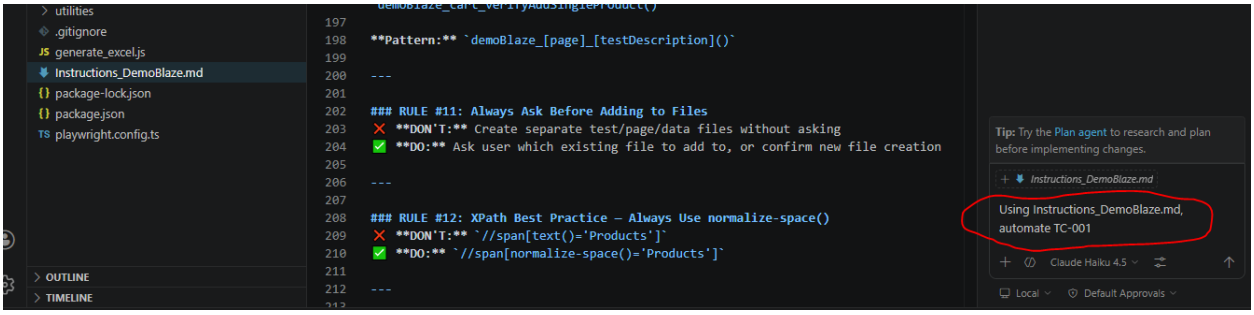
Chat Input:

```
Using Instructions_DemoBlaze.md, automate TC-001
```

Output:

```
Will provide loginPage.ts and homePageTest.spec.ts files with code.
```

Copilot Chat Prompt:

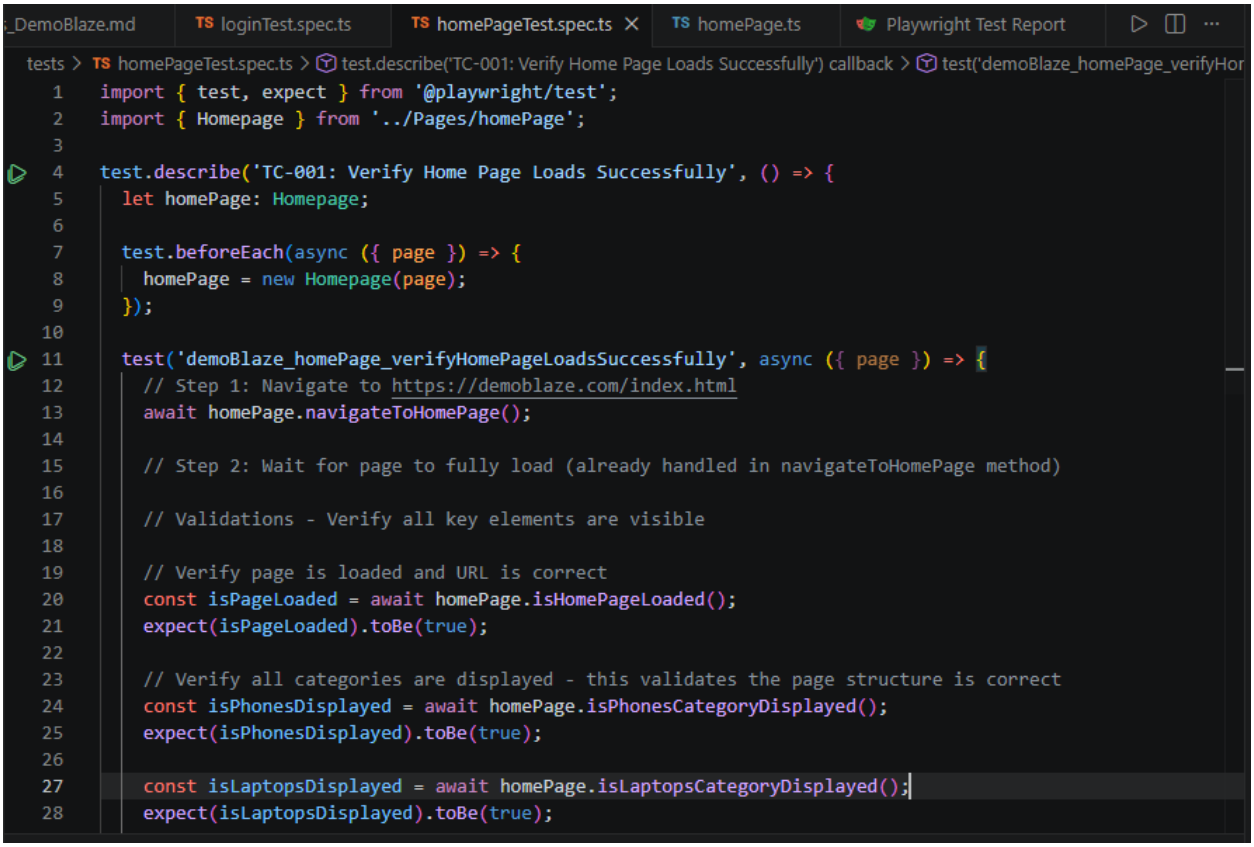


6.3 Step 3: Copilot Generates Code

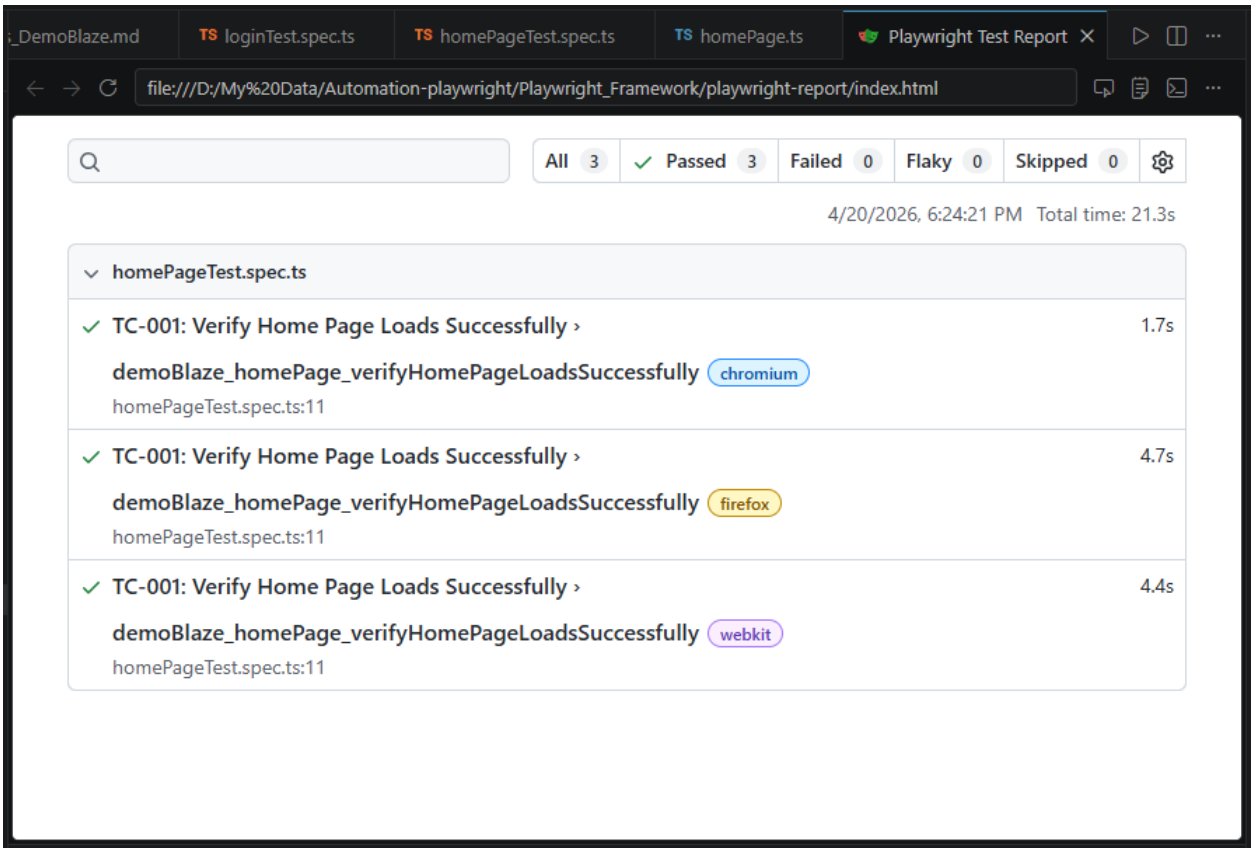
Copilot will generate:

- Login Page Object (loginPage.ts) with locators and methods
- Test Specification (homePageTest.spec.ts) with test cases
- Complete TypeScript code following all Instructions.md rules

Copilot Generated Code:



Execution Report:

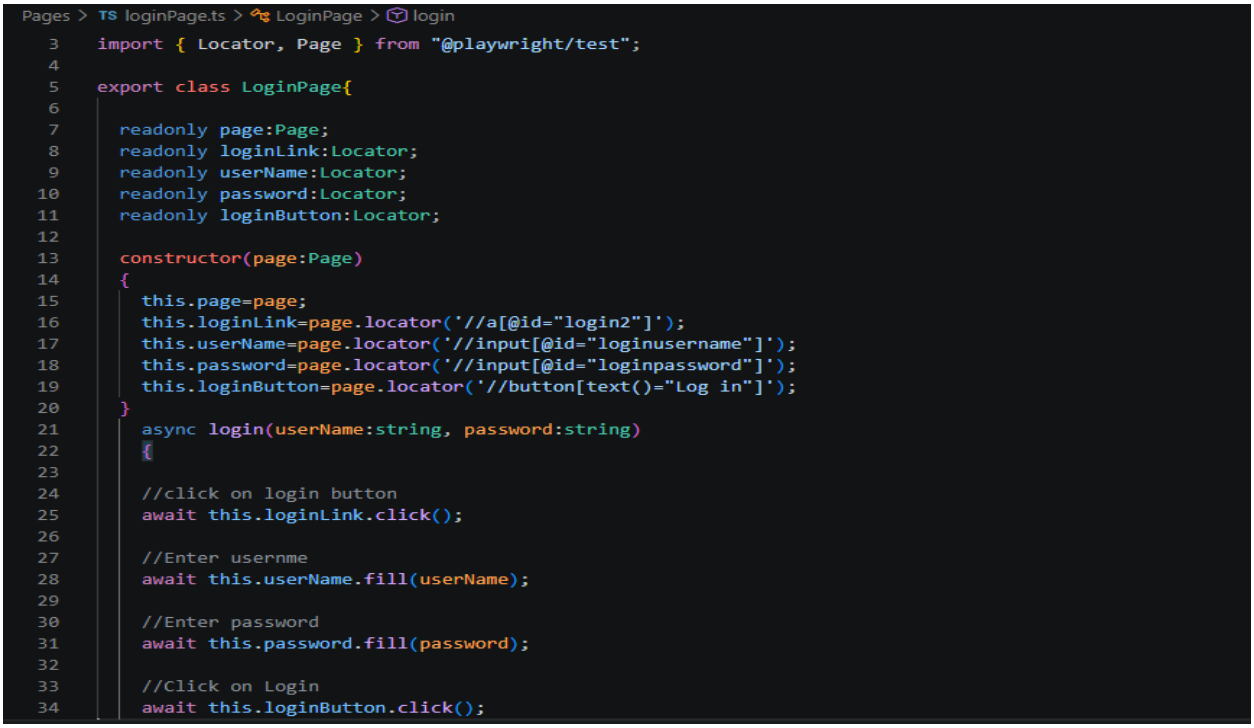


6.4 Step 4: Copy Code to Files (if the files are not created)

1. Copy the Page Object code from chat
2. Create Pages/loginPage.ts file and paste code
3. Copy the Test Specification code
4. Create tests/homePageTest.spec.ts file and paste code

Files Created in VS Code:

1. loginPage.ts



```
Pages > TS loginPage.ts > LoginPage > login
import { Locator, Page } from "@playwright/test";

export class LoginPage{

  readonly page:Page;
  readonly loginLink:Locator;
  readonly userName:Locator;
  readonly password:Locator;
  readonly loginButton:Locator;

  constructor(page:Page)
  {
    this.page=page;
    this.loginLink=page.locator('//a[@id="login2"]');
    this.userName=page.locator('//input[@id="loginusername"]');
    this.password=page.locator('//input[@id="loginpassword"]');
    this.loginButton=page.locator('//button[text()="Log in"]');
  }
    async login(userName:string, password:string)
    {

    //click on login button
    await this.loginLink.click();

    //Enter usernme
    await this.userName.fill(userName);

    //Enter password
    await this.password.fill(password);

    //Click on Login
    await this.loginButton.click();
```

2. homePageTest.spec.ts

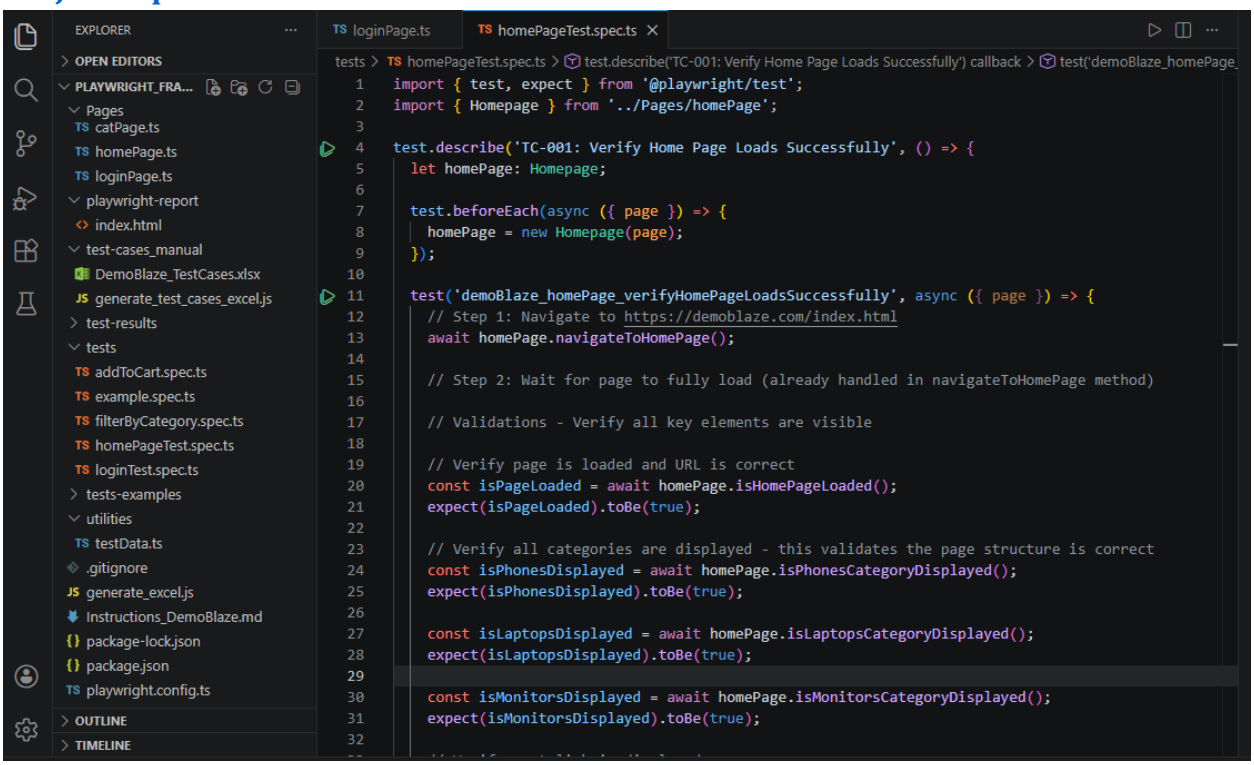
```
> test.describe('TC-001: Verify Home Page Loads Successfully') callback > test('demoBlaze_homePage_verifyHomePageLoadsSuccessfully') callback
import { test, expect } from '@playwright/test';
import { Homepage } from '../Pages/homePage';

test.describe('TC-001: Verify Home Page Loads Successfully', () => {
  let homePage: Homepage;

  test.beforeEach(async ({ page }) => {
    homePage = new Homepage(page);
  });

  test('demoBlaze_homePage_verifyHomePageLoadsSuccessfully', async ({ page }) => {
    // Step 1: Navigate to https://demoblaze.com/index.html
    await homePage.navigateToHomePage();

    // Step 2: Wait for page to fully load (already handled in navigateToHomePage method)

    // Validations - Verify all key elements are visible

    // Verify page is loaded and URL is correct
    const isPageLoaded = await homePage.isHomePageLoaded();
    expect(isPageLoaded).toBe(true);

    // Verify all categories are displayed - this validates the page structure is correct
    const isPhonesDisplayed = await homePage.isPhonesCategoryDisplayed();
    expect(isPhonesDisplayed).toBe(true);

    const isLaptopsDisplayed = await homePage.isLaptopsCategoryDisplayed();
    expect(isLaptopsDisplayed).toBe(true);

    const isMonitorsDisplayed = await homePage.isMonitorsCategoryDisplayed();
    expect(isMonitorsDisplayed).toBe(true);
```

7. Phase 5: Project Structure & Organization

7.1 Complete Folder Structure

```
Playwright_Framework/
├── Pages/
│   ├── loginPage.ts
│   ├── homePage.ts
│   └── cartPage.ts
├── tests/
│   ├── auth.spec.ts
│   ├── shopping.spec.ts
│   └── checkout.spec.ts
├── utilities/
│   ├── testData.ts
│   └── helpers.ts
├── test-cases_manual/
│   └── TestCases.xlsx
├── Instructions_DemoBlaze.md
├── playwright.config.ts
├── package.json
└── README.md
```

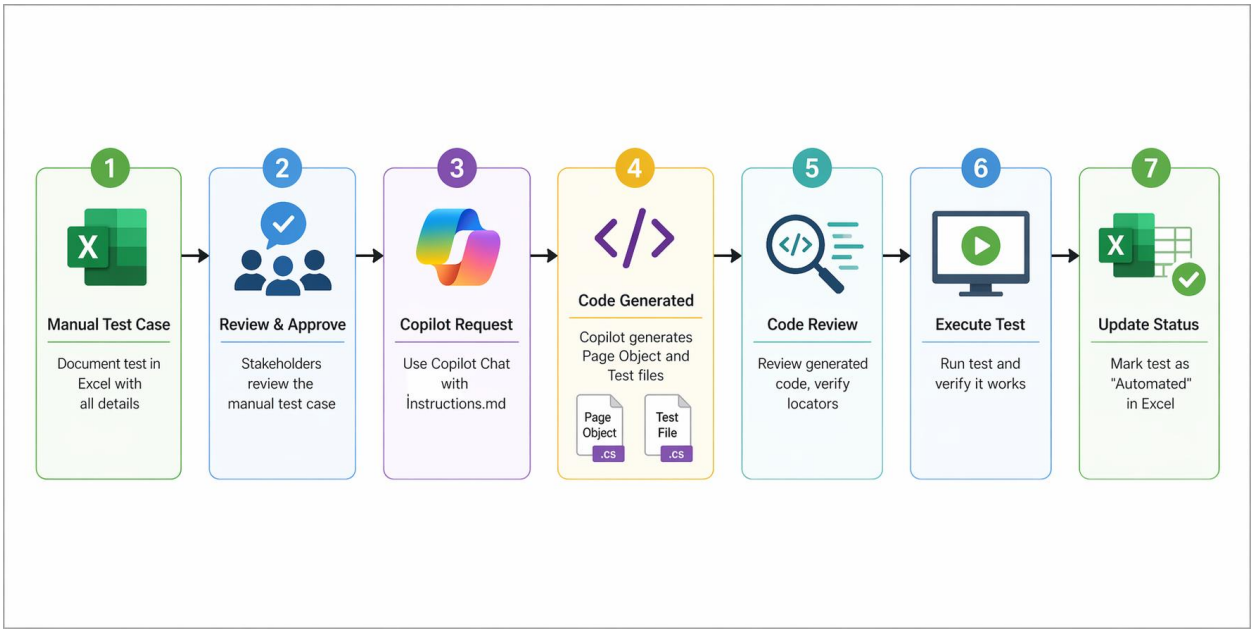
Project Explorer Structure:



8. Phase 6: Complete Automation Workflow

- 1. Manual Test Case: Document test in Excel with all details
- 2. Review & Approve: Stakeholders review the manual test case
- 3. Copilot Request: Use Copilot Chat with Instructions.md
- 4. Code Generated: Copilot generates Page Object and Test files
- 5. Code Review: Review generated code, verify locators
- 6. Execute Test: Run test and verify it works
- 7. Update Status: Mark test as "Automated" in Excel

Complete Automation Workflow:



9. Copilot Rules & Best Practices

9.1 The 10 Critical Rules Summary

- Rule 1: Always Display Test Case First:
  - Display complete test case, WAIT for "Yes, proceed" before coding
- Rule 2: Capture Both Locators:
  - Capture action locators (buttons) AND validation locators (messages)
- Rule 3: Never Guess Locators:
  - Inspect actual DOM, verify selector works before using
- Rule 4: Use Locator Priority:
  - Prefer data-testid > id > class > CSS > XPath
- Rule 5: Use Page Object Model:
  - Declare elements as properties, use methods for interactions
- Rule 6: Use Locator Suffix:
  - Element names: loginButtonLocator (not loginButton)
- Rule 7: Avoid Complex Assertions:
  - Create assertion methods in page objects, keep tests clean
- Rule 8: No Hard Waits:
  - Use waitFor() with state conditions instead of setTimeout()
- Rule 9: Implement ALL Steps:
  - Complete all test steps and validations, no partial implementations
- Rule 10: Follow Naming:
  - Pattern: demoBlaze_[page]_[description]() like demoBlaze_login_verifyValidUserLogin()

10. Testing & Execution

10.1 Essential Test Commands

All Tests:

```
npx playwright test
```

Specific File:

```
npx playwright test tests/auth.spec.ts
```

Specific Test:

```
npx playwright test -g "verify valid user"
```

Headed Mode:

```
npx playwright test --headed
```

Debug Mode:

```
npx playwright test --debug
```

Chromium Only:

```
npx playwright test --project=chromium
```

Verbose:

```
npx playwright test --verbose
```

View the generated HTML report:

```
npx playwright show-report
```

11. Troubleshooting & Tips

11.1 Common Issues and Solutions

Issue	Solution
Element not found	Check locator is correct using page.screenshot(), check DOM
Test timeout	Increase timeout in playwright.config.ts, add waitFor()
Flaky tests	Use waitFor() with state conditions, avoid setTimeout()
Copilot code incorrect	Provide detailed Instructions.md, ask Copilot to refactor
Module not found	Run npm install, check import paths in files
Tests slow	Enable parallelization, optimize waits, reduce delays
Report not generating	Verify HTML reporter in config, check file permissions

11.2 Tips for Success:

- Keep Instructions.md updated with new patterns and application selectors
- Review Copilot-generated code before using it in production
- Use meaningful variable names ending with "Locator"
- Test generated code in headed mode first (--headed flag)
- Maintain test data separately in utilities/testData.ts
- Use Page Object Model consistently across all tests
- Add comments to complex test logic
- Run tests regularly to catch failures early
- Update locators when application UI changes
- Use Git version control for all test code

Conclusion:

You now have a complete guide for building and maintaining a Playwright test automation framework with GitHub Copilot Chat assistance. This methodology enables you to:

- Create well-structured manual test cases in Excel
- Use Instructions.md to guide Copilot in code generation
- Generate consistent, maintainable automation scripts
- Execute tests and view comprehensive HTML reports
- Scale your test suite efficiently
- Maintain code quality and framework standards

Quick Start Checklist:

- ☐ Install Node.js and VS Code
- ☐ Install Playwright and dependencies
- ☐ Create project structure (Pages, tests, utilities)
- ☐ Create Instructions_DemoBlaze.md with 10 critical rules
- ☐ Create first manual test case in Excel
- ☐ Request Copilot Chat to generate automation code
- ☐ Copy generated code to Page Object and Test files
- ☐ Run: npx playwright test
- ☐ View report: npx playwright show-report
- ☐ Repeat for more test cases